

An Analytical Study of Algorithmic Solutions for *Lights Out* and *Tetravex*

Ghulam Junaid

Abstract

This paper presents an analytical study of two algorithmic solutions developed for classic combinatorial games: *Lights Out* and *Tetravex*. The *Lights Out* solution uses a breadth-first search (BFS) over a bitmask-encoded state space to guarantee optimality, while the *Tetravex* solution is formulated as a constraint satisfaction problem (CSP) and solved using depth-first search (DFS) with backtracking, constraint propagation, and heuristic ordering. We analyze correctness, time and space complexity, design decisions, limitations, and scalable improvements of both approaches.

1 Introduction

Combinatorial puzzles provide a natural setting for studying state-space search, optimization, and constraint reasoning. *Lights Out* is a deterministic finite-state puzzle with a well-defined transition graph, making it suitable for exhaustive shortest-path algorithms. *Tetravex*, in contrast, is a tile-placement puzzle characterized by strong local constraints and factorial growth of possibilities, requiring aggressive pruning and heuristics.

This paper analyzes two implementations reflecting these differences: a global BFS-based solver for *Lights Out* and a heuristic-driven DFS solver for *Tetravex*.

Implementation Availability

The complete implementations for some of the ideas of the *Lights Out* and *Tetravex* solvers, are publicly available at:

<https://github.com/GhulamJunaid07/Lights-out-and-Tetravex-solvers>

2 Lights Out

2.1 Problem Modeling

The standard 5×5 *Lights Out* board is encoded as a 25-bit integer. Each bit represents the on/off state of a cell. Clicking a cell toggles the state of that cell and its orthogonal neighbors. This induces a graph where:

- Vertices correspond to all 2^{25} possible board configurations.
- Edges correspond to a single cell click.

The objective is to reach the all-zero configuration from a given initial state.

2.2 Algorithm

The solver performs a breadth-first search starting from the zero configuration. For each state, all 25 possible clicks are explored by applying bitwise XOR operations to generate neighboring states. Distances and parent pointers are stored to reconstruct an optimal solution.

2.3 Correctness

The algorithm is correct because BFS explores states in non-decreasing order of distance, all reachable configurations are visited, and transitions are deterministic. If a configuration is unreachable, the algorithm correctly reports that no solution exists.

2.4 Complexity Analysis

Time Complexity. There are 2^{25} possible states, and each state generates 25 transitions:
 $\mathcal{O}(25 * 2^{25}) = \mathcal{O}(2^{25})$

Space Complexity. Distance and parent arrays are stored for every state:
 $\mathcal{O}(2^{25})$

2.5 Strengths and Limitations

This approach guarantees optimal solutions and allows fast query resolution after preprocessing, but requires very high memory usage and is limited to fixed board sizes.

2.6 Possible Improvements

Several improvements are possible. Bidirectional BFS and meet-in-the-middle techniques can reduce the explored state space. A more fundamental optimization is to reformulate the problem using linear algebra over $\mathbb{F}_2$, eliminating the need for full state-space traversal.

2.7 Row-Reduction Approach for Larger Boards

A scalable extension is to reduce the effective state space by eliminating rows sequentially. Starting from the top row, moves can be chosen to turn off all illuminated cells in the current row, propagating their effects only to the row below. Repeating this process reduces any configuration to one where all active cells lie in a single designated row.

After this reduction, the entire board state is determined by the final row, yielding only 2^n possible configurations for an $n \times n$ board. A secondary search or precomputation can then determine whether a given bottom-row configuration is solvable. This reduces the problem from 2^{n^2} states to 2^n states and scales significantly better for larger boards. Conceptually, this approach is closely related to Gaussian elimination over $\mathbb{F}_2$.

3 Tetravex

3.1 Problem Modeling

Tetravex consists of an $n \times n$ grid and n^2 unique tiles. Each tile has four labeled edges (top, right, bottom, left). A valid solution places all tiles such that adjacent edges match exactly. This problem can be modeled as a constraint satisfaction problem with spatial and combinatorial constraints.

3.2 Algorithm

The solver uses depth-first search with backtracking. Tiles are placed starting from the center of the board and expanding outward. At each step, feasible tiles are filtered based on already placed neighbors, and the algorithm recursively explores valid placements.

3.3 Correctness

The algorithm only places tiles that satisfy all local constraints and uses backtracking to explore all valid configurations. A solution is recorded only when all grid positions are filled, ensuring correctness.

3.4 Complexity Analysis

Time Complexity. In the worst case, the time complexity is factorial:

$$\mathcal{O}((n^2)!)$$

In practice, constraint propagation and heuristics drastically reduce the search space.

Space Complexity. The algorithm uses $\mathcal{O}(n^2)$ memory for the board, recursion stack, and auxiliary data structures.

3.5 Row-Based Decomposition Approach

An improvement is to decompose the board row-wise. All valid row configurations are first generated using a DFS that constructs sequences of n tiles whose horizontal edges match. Each row uses distinct tiles.

A second DFS then determines a vertical ordering of these rows. Two rows are compatible if, for every column j , the bottom edge of the upper row matches the top edge of the lower row. This reduces the problem to selecting n compatible rows that collectively use all n^2 tiles exactly once.

This approach reduces recursion depth from n^2 to n , strengthens constraints early, and significantly reduces the branching factor. It closely resembles layered CSP and exact-cover formulations.

4 Author’s Insight and Lessons Learned

Developing these solvers highlighted the importance of modeling choices in combinatorial problems. While brute-force search can be effective for small state spaces, scalable solutions often require restructuring the problem itself. Decomposing a search space along natural dimensions, such as rows or linear constraints, can yield exponential reductions in complexity. These insights guided the proposed improvements and reflect a broader principle applicable to many algorithmic problems.

5 Conclusion

The two solutions illustrate contrasting but appropriate algorithmic paradigms. Lights Out benefits from full state-space traversal and guarantees optimality, while Tetravex relies on heuristic-guided search and constraint reasoning to manage combinatorial explosion. With further refinement, both approaches can be extended into scalable and highly optimized solvers.